engineering proceedings

MDPI

Proceeding Paper

Tracert-Retrieval-Augmented Generation: Boosting Multi-Hop Retrieval-Augmented Generation with Direction-Aware Graph Traversal †

Siu-Him Zhang [1] and Jhe-Wei Lin [2,*]

[1] iSchool, Feng Chia University, Taichung 407102, Taiwan; d1177531@o365.fcu.edu.tw
[2] Department of Information Engineering and Computer Science, Feng Chia University, Taichung 407102, Taiwan
* Correspondence: jhewlin@fcu.edu.tw
† Presented at 8th International Conference on Knowledge Innovation and Invention 2025 (ICKII 2025), Fukuoka, Japan, 22–24 August 2025.

Abstract

Tracert-retrieval-augmented generation (RAG) is a novel retrieval-augmented framework designed for efficient, document-level multi-hop reasoning. Unlike conventional RAG systems that retrieve top-*k* text segments based solely on dense similarity, Tracert-RAG predicts a semantic goal vector from the user query, constructs a local semantic graph from the document embeddings, and employs a direction-aware greedy traversal to identify reasoning paths toward the goal. This system eliminates the inflexibility of symbolic graph traversal and the inefficiency of manual query reformulation. On literary analysis tasks from Pride and Prejudice, Tracert-RAG outperforms standard RAG and graph RAG baselines in answer quality, inference speed, and interpretability. Specifically, it achieves the highest average answer quality (8.05 out of 10) while reducing indexing time by a factor of 80 compared to graph RAG methods.

Keywords: retrieval-augmented generation; multi-hop reasoning; semantic graph traversal; goal vector prediction; document-level inference

Academic Editors: Teen-Hang Meen, Chun-Yen Chang and Cheng-Fu Yang

Published: 5 February 2026

Citation: Zhang, S.-H.; Lin, J.-W. Tracert-Retrieval-Augmented Generation: Boosting Multi-Hop Retrieval-Augmented Generation with Direction-Aware Graph Traversa. *Eng. Proc.* **2025**, *120*, 47. https://doi.org/10.3390/engproc2025120047

1. Introduction

Retrieval-augmented generation (RAG) has become a powerful framework for language models in external knowledge. However, traditional RAG systems typically rely on top-*k* similarity-based retrieval, which often returns fragmented and loosely connected passages. This hinders the model's capacity for multi-hop reasoning and coherent document-level understanding.

To overcome these limitations, Tracert-RAG redefines retrieval as a goal-directed traversal problem within the embedding space. In this study, we constructed a semantic graph that connects the user query to a predicted goal vector representing the user's latent intent. A direction-aware traversal algorithm then identifies a coherent reasoning path through semantically aligned nodes. This process enables contextually rich, interpretable, and efficient generation at the document level. In addition, by eliminating the need for brittle symbolic graphs and manual query rewriting, Tracert-RAG provides a scalable and robust solution for complex question-answering tasks.

2. Related Work

2.1. Native RAG

RAG combines dense retrieval with neural generation by retrieving the top-k semantically similar fragments from a corpus and passing them to a language model to generate responses [1,2]. This setup relies on vector similarity (e.g., cosine distance) and uses a vector database to store and query embedded text chunks [3–6].

While simple and efficient, native RAG systems often struggle with multi-hop reasoning, where answering a query requires combining evidence from non-adjacent parts of the source text. The top-k retrieval frequently selects redundant or narrowly focused thematic content, limiting the model's ability to synthesize global insights [7,8].

2.2. Graph RAG

Graph RAG extends the RAG framework by constructing a symbolic knowledge graph over the input corpus. Entities and relationships are extracted using large language models or external tools, then clustered into communities with associated summaries. Retrieval is performed hierarchically across this graph, allowing for better global semantic coverage and improved answer grounding in structured evidence [9,10]. Compared with the original RAG, graph RAG provides more interpretable and coherent responses [11], especially for complex or multifaceted queries. However, this approach incurs high preprocessing and inference costs and depends heavily on external symbolic infrastructure.

2.3. Geometric Representations of Queries

Geometric query models such as Query2Box [12,13] and BetaE [14–16] represent logical queries over knowledge graphs [17] as bounded regions in embedding space. While effective for symbolic multi-hop inference, these models are designed for structured triple-based knowledge graphs and do not apply to free-form natural language corpora or retrieval-augmented generation settings.

3. Methodology

3.1. Tracert-RAG Pipeline

Tracert-RAG enhances traditional RAG by replacing static top-k nearest neighbor retrieval with a dynamic goal-directed traversal over a semantic graph [18,19] constructed in embedded space. The pipeline consists of indexing and query-time retrieval and generation phases.

3.1.1. Indexing Phase

In document chunking, the corpus is divided into fixed-size overlapping text segments [20,21]. In embedding, each chunk is embedded using a fixed embedding model [22,23]. The embedded chunks are inserted into a vector database [24] for efficient retrieval in indexing.

3.1.2. Query-Time Retrieval and Generation Phase

1. In goal vector prediction, a user query q is processed by a large language model (LLM), which generates a reformulated version q' that captures the expected answer or deeper intent. This reformulated query is then embedded to produce the goal vector g. In query vector embedding, the original query q is embedded into a start vector q v, which serves as the initial point for semantic navigation. During radius-based filtering, the semantic $d = ||v_g - v_q||$ between the goal and query vectors is calculated. This distance is used as a dynamic radius to select a subset of document

chunks—previously indexed during the chunking and embedding phase—that fall within this semantic corridor.

2. In direction-aware graph construction, a graph is built over the filtered document chunks. The edges in this graph are weighted based on both cosine similarity and their alignment with the directional vector from the query to the goal. Paths that deviate from this intended trajectory are penalized to maintain semantic relevance. Finally, in greedy path traversal, a lightweight, directional greedy algorithm is used to traverse the graph from the start vector v_q to the goal vector v_g. At each step, the algorithm selects the next node based on corrected similarity scores, ensuring an efficient and contextually aligned retrieval path.

3.2. Indexing

In the indexing phase, raw documents are first preprocessed and segmented into semantically meaningful units that are suitable for downstream embedding and retrieval. To ensure both contextual completeness and boundary coherence, each document is divided into overlapping text chunks of fixed length—for example, 500 characters with a 100-character overlap. This overlapping strategy helps mitigate the risk of splitting semantically related content across chunk boundaries, thereby preserving local narrative continuity.

Each chunk is then passed through a pre-trained sentence embedding model, such as text-embedding-ada-002, to generate dense vector representations that capture its semantic content in high-dimensional space. These embeddings, along with metadata such as source identifiers and positional indices, are stored in a vector database like Milvus. This setup enables efficient similarity-based search and forms the foundation for the semantic retrieval process in Tracert-RAG.

3.3. Goal Vector Prediction

In contrast to native RAG systems that rely solely on similarity-based retrieval, Tracert-RAG incorporates a goal vector to guide the retrieval process toward a semantically aligned endpoint. Given a user query q, we prompt a large language model (LLM) to generate a rephrased or projected variant of the query, denoted as q'. This reformulated query is not intended to provide a direct answer but rather to serve as an inferred abstraction that better captures the user's latent information need. In the context of our framework, q' acts as a semantic anchor or destination for guiding the traversal process.

Both q and q' are embedded into the same latent vector space using a shared embedding model (e.g., text-embedding-ada-002). These yield the following.

- $v_q \in R^d$: the query vector (start).
- $v_g \in R^d$: the predicted goal vector (end).

The goal vector v_g serves as the destination node in the semantic traversal graph and plays a pivotal role in guiding the retrieval process. By providing a clear semantic objective, it helps define the trajectory along which relevant document chunks are selected, ensuring that the retrieved content aligns with the user's underlying intent rather than surface-level similarity. To generate the reformulated query q', we utilize the GPT-4o-mini language model. This projection step translates the original query into a more goal-oriented representation that serves as the semantic target for traversal.

The example of the output is as follows. Given the input question, "How does Elizabeth's initial reaction to Mr. Darcy at the Meryton assembly (Chapter 3) contrast with her response when he proposes to her (Chapter 34), and what key events in between shape that change of heart?", the model replies as follows.

Category: Literary Analysis
Concept: Character Development (Explanation: The contrast in Elizabeth's reactions to Mr. Darcy illustrates her growth and changing perceptions influenced by key events, such as his initial arrogance, the revelations about Wickham, and Darcy's later actions that demonstrate his true character.)

We embedded the full Concept + Explanation field as the goal vector q', which anchors the direction of traversal.

3.4. Radius-Based Filtering

To retrieve the documents, we use the cosine distance as the radius r. Given the query embedding v_q, we retrieve only those document vectors v_d as follows.

$$r = 1 - cos(v\_q,\ v\_g) \tag{1}$$

We then retrieve a subset of document vectors $\mathcal{V}_r \subset \mathcal{V}$ from the embedding index to obtain Equation (2).

$$\mathcal{V}_r = \{v_d \in \mathcal{V} \mid 1 - \cos(v_q, v_d) \leq r\} \tag{2}$$

This constraint helps eliminate semantically irrelevant content, thereby reducing retrieval noise and ensuring that the constructed graph is composed of document segments that are plausibly aligned with both the user's original query and the inferred goal vector. In practical terms, this radius-based filtering significantly narrows down the candidate node set from the full corpus to a more manageable and computationally efficient subset, while still preserving sufficient semantic coverage for effective downstream reasoning.

3.5. Direction-Aware Graph Construction

We construct a graph $G = (V, E)$, where each node represents a document chunk embedded in a high-dimensional semantic space. Edges are added between nodes whose cosine distance falls below a fixed threshold δ. Each edge $(u, v) \in E$ is assigned a directional cost based on both local similarity and semantic alignment with the global query-to-goal direction.

The cost function is defined as follows.

$$\text{cost}(u, v) = \text{cosine\_distance}(v_u, v_v) + \alpha \cdot D(u \to g) \tag{3}$$

- $v_u,\ v_v$: embedding vectors of nodes u and v.
- v_g: goal vector (prediction by the LLM).
- $D(u \to g)$: angular penalty measuring divergence between the direction from v_u to, v_v and the direction from v_u to the goal v_g.
- α: tunable penalty weight (typically between 0.1 and 0.5).
- We define the angular penalty as follows.

$$D(u \to g) = 1 - \cos(\theta) = 1 - \frac{(v_v - v_u) \cdot (v_g - v_u)}{|v_v - v_u| \cdot |v_g - v_u|} \tag{4}$$

This equation is used to measure how well the edge $(u,\ v)$ aligns with the overall semantic direction toward the goal v_g. A smaller value indicates that the transition from u to v is more 'on-track' with the intended semantic trajectory; a larger value reflects a semantic detour.

By including this penalty, we bias the graph traversal to favor a forward semantic trajectory rather than drifting laterally or looping around semantically similar but uninformative chunks. The strength of this bias is controlled by the hyperparameter α, which can

be tuned to balance local similarity against global goal alignment. Algorithm 1 summarizes the computation of the directional penalty $D(u \to g)$ used in the edge cost formulation.

Algorithm 1: directional penalty computation

The angular penalty function is used with the following.

1. Input:
- v_u: Embedding of current node
- v_v: Embedding of neighbor candidate
- v_g: Embedding of goal node

2. Output:

Scalar penalty value $D(u \to v, v_g) \in [0, 2]$

3.6. Greedy Path Traversal

To generate the traversal path, The overall direction-aware greedy traversal procedure is formalized in Algorithm 2. Starting from the query vector v_q, the algorithm iteratively selects the next node with the lowest corrected cost until the goal vector v_g is reached or no valid transition exists. At each step, the algorithm selects the next unvisited neighbor node with the lowest corrected edge cost [25]. This cost includes local cosine similarity and angular alignment toward the goal vector, encouraging the path to move semantically forward. Different from more global search algorithms such as A* or beam search, the method in this study is intentionally lightweight and deterministic. It performs a single pass without backtracking or maintaining candidate queues, which makes it highly efficient while preserving semantic relevance.

To guide the traversal toward meaningful answer content, we apply a directional penalty during graph construction. This penalty increases the cost of edges that deviate from the semantic direction defined by the vector $v_q \to v_g$. The corrected edge cost is defined as follows.

$$\text{cost}(u, v) = \text{cosine\_distance}(v_u, v_v) + \alpha \cdot D(u \to v, v_g) \tag{5}$$

$$D(u \to v, v_g) = 1 - \frac{(v_v - v_u) \cdot (v_g - v_u)}{|v_v - v_u| \cdot |v_g - v_u|} \tag{6}$$

Here, α is a tunable weight, and $D(\cdot)$ is the angular penalty function.

Algorithm 2: direction-aware greedy path traversal

The greedy algorithm for direction-aware traversal is created using the following.

1. Input
- $G = (V, E)$: Graph with corrected edges costs
- v_q: Query vector mode (start)
- v_g: Goal vector node (end)
- α: Directional penalty weight

2. Output
- Path from v_q to v_g, or None if no path is found

4. Results

4.1. Experiment

We evaluated the effectiveness of Tracert-RAG using a domain-specific benchmark derived from 'Pride and Prejudice' by Jane Austen. The novel consists of five manually curated, multi-hop literary analysis questions, each requiring the model to perform causal

reasoning, belief revision, or tracking of character development across non-contiguous narrative segments [26].

We compared Tracert-RAG against the following two retrieval baselines.

- Native RAG [27]: A standard top-*k* dense retriever that selects the most similar document chunks based on cosine similarity between the user query and embedded document segments. This approach lacks any notion of structure or intermediate reasoning steps.
- Graph RAG [28]: A hierarchically structured retrieval method introduced by Microsoft Research. It extracts a symbolic knowledge graph from raw text, builds community hierarchies from this graph, and generates community-level summaries to guide retrieval. This enables structured, multi-resolution retrieval that captures global topic organization more effectively than native semantic search.

4.2. Quantitative Result

We evaluated system performance across five benchmark questions, using a rubric consisting of four criteria: comprehensiveness, textual engagement, analytical depth, and clarity. Each answer is rated on a 1–10 scale for each dimension. The scoring process was conducted using the GPT-4o model, which was also instructed to determine the best overall answer for each question based on aggregate quality. Table 1 presents the average scores for each evaluation dimension across all questions, as well as the number of questions for which each method achieved the highest overall ranking.

Table 1. Average scores and question wins per method. Tracert-RAG ranks highest overall across 3 of 5 benchmark questions.

Method	Win	Average Comprehensiveness	Average Engagement	Average Depth	Average Clarity	Average Overall
Graph RAG	2	8.2	7.4	8.2	7.4	7.75
Tracert TAG	3	8.2	8.0	7.8	8.2	8.05
Native RAG	0	6.6	6.2	6.6	7.0	6.65

Tracert-RAG achieves the highest overall score (8.05) and is ranked best on 3 of the 5 benchmark questions. It performs consistently across categories, with particularly strong performance in Clarity and Engagement. Graph RAG demonstrates strength in capturing global thematic structure but often includes redundant or loosely relevant content. Native RAG lags across all dimensions due to its inability to support multi-step or structured reasoning.

4.3. Efficiency

As summarized in Table 2, Tracert-RAG achieves more than an 80× speedup in indexing time compared to Graph RAG (approximately 2.5 min versus 3 h) and reduces the average per-question inference time by about 6× (11 s vs. 92.6 s). Although Tracert-RAG is moderately slower than Native RAG at runtime (approximately 2.5× slower), the additional computational cost enables multi-hop reasoning, making it a practical solution for real-time applications.

Table 2. Time consumption by retrieval method.

Method	Indexing Time(s)	Query + Answer Time(s)	Avg. Total Per-Query Time(s)
Graph RAG	11788.01	463.04	~92.6
Tracert TAG	147.08	56.89	~11.37
Native RAG	163.79	29.98	~5.99

5. Conclusions

We developed Tracert-RAG, a retrieval-augmented generation framework that enables efficient multi-hop reasoning through direction-aware graph traversal in embedding space. On a literary benchmark, it outperformed Native and graph RAG in answer quality (8.05/10) while achieving 80× faster indexing and low per-query latency (11 s). Balancing interpretability, speed, and accuracy, Tracert-RAG is a practical solution for real-time, reasoning-intensive applications. Future work will explore open-domain QA and hybrid symbolic integration.

Author Contributions: Methodology, S.-H.Z. and J.-W.L.; software, S.-H.Z.; validation, S.-H.Z. and J.-W.L.; formal analysis, S.-H.Z. and J.-W.L.; writing—original draft preparation, S.-H.Z. and J.-W.L.; writing—review and editing, J.-W.L.; funding acquisition, J.-W.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the National Science and Technology Council, Taiwan, under grant 113-2221-E-035 -059 -.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data will be made available on reasonable request.

Acknowledgments: The authors would like to acknowledge the support of the National Science and Technology Council (NSTC), Taiwan. Special thanks are also extended to the Department of Computer Science and Engineering at Feng Chia University for providing laboratory resources and computing facilities essential to this research.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.T.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In Proceedings of the 34th International Conference on Neural Information Processing Systems, Vancouver, Canada, 6–12 December 2020; pp. 9459–9474.
2. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.S.; Wu, L.; Edunov, S.; Chen, D.; Yih, W.T. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), online, 16–20 November 2020; pp. 6769–6781. [CrossRef]
3. Yang, S.; Gribovskaya, E.; Kassner, N.; Geva, M.; Riedel, S. Do Large Language Models Latently Perform Multi-Hop Reasoning? *arXiv* **2025**, arXiv:2402.16837.
4. Yu, S.; Kim, I.-H.; Song, J.; Lee, S.; Park, J.; Yoon, S. Unleashing Multi-Hop Reasoning Potential in Large Language Models through Repetition of Misordered Context. *arXiv* **2025**, arXiv:2410.07103.
5. Zhang, M.; Fang, B.; Liu, Q.; Ren, P.; Wu, S.; Chen, Z.; Wang, L. Enhancing Multi-hop Reasoning through Knowledge Erasure in Large Language Model Editing. *arXiv* **2024**, arXiv:2408.12456. [CrossRef]
6. Zhao, W.; Chiu, J.T.; Cardie, C.; Rush, A.M. HOP, UNION, GENERATE: Explainable Multi-hop Reasoning without Rationale Supervision. *arXiv* **2023**, arXiv:2305.14237. [CrossRef]
7. Jiang, P.; Cao, L.; Zhu, R.; Jiang, M.; Zhang, Y.; Sun, J.; Han, J. RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation. *arXiv* **2025**, arXiv:2502.10996.
8. Lin, J.; Liu, J.; Liu, Y. Optimizing Multi-Hop Document Retrieval Through Intermediate Representations. *arXiv* **2025**, arXiv:2503.04796.
9. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R.O.; Larson, J. From Local to Global: A graph RAG Approach to Query-Focused Summarization. *arXiv* **2024**, arXiv:2404.16130. [CrossRef]
10. Han, H.; Wang, Y.; Shomer, H.; Guo, K.; Ding, J.; Lei, Y.; Halappanavar, M.; Rossi, R.A.; Mukherjee, S.; Tang, X.; et al. Retrieval-Augmented Generation with Graphs (GraphRAG). *arXiv* **2025**, arXiv:2501.00309.
11. Xiang, Z.; Wu, C.; Zhang, Q.; Chen, S.; Hong, Z.; Huang, X.; Su, J. When to use Graphs in RAG: A Comprehensive Analysis for Graph Retrieval-Augmented Generation. *arXiv* **2025**, arXiv:2506.05690. [CrossRef]
12. Ren, H.; Hu, W.; Leskovec, J. Query2box: Reasoning over knowledge graphs in vector space using box embeddings. *arXiv* **2020**, arXiv:2002.05969. [CrossRef]

13. Zhapa-Camacho, F.; Hoehndorf, R. Fully Geometric Multi-Hop Reasoning on Knowledge Graphs with Transitive Relations. *arXiv* **2025**, arXiv:2505.12369. [CrossRef]
14. Ren, H.; Leskovec, J. Beta embeddings for multi-hop logical reasoning in knowledge graphs. In Proceedings of the 34th International Conference on Neural Information Processing Systems, Vancouver, Canada, 6–12 December 2020; pp. 19716–19726.
15. Kim, J.; Jung, H.; Jang, H.; Park, H. Improving Multi-hop Logical Reasoning in Knowledge Graphs with Context-Aware Query Representation Learning. *arXiv* **2024**, arXiv:2406.07034.
16. Fei, W.; Wang, Z.; Yin, H.; Zhao, S.; Zhang, W.; Song, Y. Efficient and Scalable Neural Symbolic Search for Knowledge Graph Complex Query Answering. *arXiv* **2025**, arXiv:2505.08155. [CrossRef]
17. Jiang, X.; Xu, C.; Shen, Y.; Sun, X.; Tang, L.; Wang, S.; Chen, Z.; Wang, Y.; Guo, J. On the Evolution of Knowledge Graphs: A Survey and Perspective. *arXiv* **2025**, arXiv:2310.04835.
18. Zhu, Z.; Hu, T.; Zhang, H.; Yang, D.; Chen, H.; Zhang, M.; Chen, X. Conversational Intent-Driven GraphRAG: Enhancing Multi-Turn Dialogue Systems through Adaptive Dual-Retrieval of Flow Patterns and Context Semantics. *arXiv* **2025**, arXiv:2506.19385.
19. Hallgarten, M.; Stoll, M.; Zell, A. From Prediction to Planning With Goal Conditioned Lane Graph Traversals. *arXiv* **2023**, arXiv:2302.07753. [CrossRef]
20. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, M.; Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv* **2023**, arXiv:2312.10997.
21. Singh, I.S.; Aggarwal, R.; Allahverdiyev, I.; Taha, M.; Akalin, A.; Zhu, K.; O'Brien, S. ChunkRAG: Novel LLM-Chunk Filtering Method for RAG Systems. *arXiv* **2025**, arXiv:2410.19572.
22. Wang, L.; Yang, N.; Huang, X.; Jiao, B.; Yang, L.; Jiang, D.; Majumder, R.; Wei, F. Text Embeddings By Weakly-Supervised Contrastive Pre-Training. *arXiv* **2024**, arXiv:2212.03533. [CrossRef]
23. Gao, A.K. Vec2Vec: A Compact Neural Network Approach for Transforming Text Embeddings with High Fidelity. *arXiv* **2023**, arXiv:2306.12689. [CrossRef]
24. Ma, L.; Zhang, R.; Han, Y.; Yu, S.; Wang, Z.; Ning, Z.; Zhang, J.; Xu, P.; Li, P.; Ju, W.; et al. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge. *arXiv* **2025**, arXiv:2310.11703.
25. Meng, S.; Wang, Y.; Yang, C.-F.; Peng, N.; Chang, K.-W. LLM-A*: Large Language Model Enhanced Incremental Heuristic Search on Path Planning. *arXiv* **2025**, arXiv:2407.02511.
26. Yu, H.; Gan, A.; Zhang, K.; Tong, S.; Liu, Q.; Liu, Z. Evaluation of Retrieval-Augmented Generation: A Survey. *arXiv* **2025**, arXiv:2405.0743.
27. Wang, Y.; Hernandez, A.G.; Kyslyi, R.; Kersting, N. Evaluating Quality of Answers for Retrieval-Augmented Generation: A Strong LLM Is All You Need. *arXiv* **2024**, arXiv:2406.18064. [CrossRef]
28. Simon, S.; Mailach, A.; Dorn, J.; Siegmund, N. A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation. *arXiv* **2024**, arXiv:2410.08801. [CrossRef]